

Transport and diffusion with finite differences

HEROIC: Advanced Topics in Computational Cardiology | Fall 2024

Zan Ahmad · Johns Hopkins University

Selected and revised from the asynchronous Python tutorial for CFD.

Finite differences connect a differential equation to a numerical experiment. This selected tutorial studies two distinct processes: transport moves a profile; diffusion smooths it. The examples use Python 3, NumPy, and Matplotlib, and contain only synthetic data.

1. Linear transport

For a constant speed $c > 0$, the equation $u_t + c u_x = 0$ translates an initial profile to the right. On an unbounded domain, the exact solution is $u(x,t) = u_0(x - ct)$. On our finite interval, we must also prescribe the value entering at the left boundary.

Let $x_i = i \Delta x$ and $t_n = n \Delta t$. A forward difference in time and a backward (upwind) difference in space give:

$$u_i^{n+1} = u_i^n - \lambda (u_i^n - u_{i-1}^n), \lambda = c \Delta t / \Delta x$$

For this scheme, use $0 \leq \lambda \leq 1$. This restriction is part of the numerical method, not an optional plotting preference. If the wave speed is negative, the upwind direction must change.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0.0, 2.0, 81)
dx = x[1] - x[0]
c = 1.0
dt = 0.5 * dx / c
steps = 40
initial = np.where((x >= 0.5) & (x <= 1.0), 2.0, 1.0)
u = initial.copy()
lam = c * dt / dx
assert 0.0 <= lam <= 1.0

for _ in range(steps):
    old = u.copy()
    u[1:] = old[1:] - lam * (old[1:] - old[:-1])
    u[0] = 1.0 # prescribed inflow; right end is outflow
```

Keep the previous time level in a separate array. Updating in place from left to right would accidentally mix old and new values and change the method.

2. Inspect transport error

```
t = steps * dt
exact = np.where((x-c*t >= 0.5) & (x-c*t <= 1.0),
                2.0, 1.0)
plt.plot(x, initial, "--", label="initial")
plt.plot(x, exact, label="translated profile")
plt.plot(x, u, label="upwind approximation")
plt.xlabel("x")
plt.ylabel("u")
plt.legend()
plt.show()
```

At this final time the transported pulse has not reached the outflow boundary. Compare its predicted location and edge shape. First-order upwinding generally smears sharp edges through numerical diffusion; this should not be confused with physical diffusion in the model.

3. Physical diffusion

The diffusion equation is $u_t = \nu u_{xx}$, where $\nu > 0$ is a diffusivity. A centered second difference gives:

$$v_i^{n+1} = v_i^n + r (v_{i+1}^n - 2v_i^n + v_{i-1}^n)$$

$$r = \nu \Delta t / \Delta x^2, \quad 0 \leq r \leq 1/2$$

Use the same spatial grid and initial hat profile, but hold both endpoint values at 1. The explicit scheme requires a time step proportional to Δx^2 .

```
nu = 0.3
dt_diff = 0.2 * dx**2 / nu
r = nu * dt_diff / dx**2
v = initial.copy()
assert 0.0 <= r <= 0.5

for _ in range(100):
    old = v.copy()
    v[1:-1] = old[1:-1] + r * (
        old[2:] - 2.0*old[1:-1] + old[:-2]
    )
    v[[0, -1]] = 1.0

plt.plot(x, initial, "--", label="initial")
plt.plot(x, v, label="diffusion")
plt.xlabel("x")
plt.ylabel("u")
plt.legend()
plt.show()
```

Experiments

Translation. Check that the pulse moves by ct . Vary λ while keeping the final physical time fixed. Describe how the edge smearing changes.

Refinement. Double the spatial resolution. Reduce Δt appropriately and compare at the same physical time. For diffusion, halving Δx requires roughly four times as many steps when r is held fixed.

Constant solutions. Replace the hat profile by $u = 1$ everywhere. Both numerical updates should preserve this constant, including the boundaries.

Stability. In a separate experiment, deliberately exceed the stated restriction. Record the first sign of nonphysical behavior. Restore stable parameters before making accuracy comparisons.

What this sample does and does not cover

The full archived tutorial also introduced nonlinear convection and Burgers' equation. This public selection focuses on the two linear problems so that the stencil, boundary conditions, stability restriction, and physical-time comparison are explicit. Extending a method to shocks requires additional care; copying a linear stencil is not a general shock solver.

Attribution and revision

Course adaptation by Zhan Ahmad. The sequence and hat-profile exercises follow Lorena A. Barba and Gilbert F. Forsyth, *CFD Python: the 12 steps to Navier-Stokes equations*, Journal of Open Source Education 1(9), 21 (2018), doi:10.21105/jose.00021.

Copyright (c) Barba group. Upstream materials: github.com/barbagroup/CFDPython. The project README specifies CC BY 4.0 for instructional content; the historical LICENSE also references CC BY 3.0. Code is BSD 3-Clause; the accompanying file retains the notice. This October 2026 selection rewrites the explanation, uses vectorized updates, adds explicit boundaries/stability checks, and corrects the discussion of numerical diffusion. No endorsement by the original authors is implied.

Download the runnable companion at zanahmad.com/files/teaching-samples/heroic-finite-differences.py.